nebio: New Hardware Handling (Step 1)

QB Nebulae V2 (RPi4) · branch **feature_newHardWareHandling** · commit 8ec5d91 · 2026-10-16

1. Summary

All hardware handling (buttons, gates, knobs, CV, encoders, LEDs) and all communication with SuperCollider and Pure Data has moved from Python into a C++ shared library, **libnebio.so**. It is loaded into the main Python program (nebulae.py) with **ctypes**. No extra daemon or process is added. Python keeps all logic (modes, ControlHandler, ui.py LED logic); it only reads inputs and sets LEDs through the library.

Step 1 keeps today's default behaviour for **Csound, Pure Data and SuperCollider**. Existing instruments, NebInterface.sc, .scd files and PD patches need no changes. The design leaves room for step 2 (sclang owns the logic and all LEDs), which is not implemented yet.

Status

Item	Status
C++ library, Python wrappers, build script	Done, committed by you as 8ec5d91
Compile (x86_64, gcc 16, all warnings on)	0 warnings, 37 exported functions
Logic tests (debounce, encoder, short presses)	0 failures
Python ↔ C++ tests (OSC, PD, init without hardware)	22 / 22 passed
Build and run on the RPi4 (aarch64)	**Not done yet**: needs testing on the device

2. Architecture

```
nebulae.py (Python, one process)
  ControlHandler / ui.py / modes          <- logic unchanged
  shiftregister.py switch.py MCP3208.py
  encoder.py scsender.py pdsender.py      <- thin wrappers
           |  ctypes (nebio.py)
           v
libnebio.so (C++17, threads inside the same process)
  input thread 1 kHz   : shift register, clicks, encoders, ADC (500 Hz)
  edge thread          : gate inputs by interrupt (libgpiod edge events)
  LED writer thread    : PCA9685 over I2C, boot/loading animations
  reset LED thread     : software PWM 150 Hz on GPIO 12
  OSC (liblo)          : send :3010, listen :3011     -> sclang
  PD (TCP)             : 127.0.0.1:3000, async connect -> Pure Data
```

C++ threads never call into Python. Python asks for a **snapshot** once per loop (updateAll()) and reads a consistent picture of all inputs. LED changes are staged and become visible when ui.py commits them, which preserves the old behaviour where the LEDs freeze during reload.

3. The C++ library (Code/nebio/)

File	What it does
include/nebio.h	The C API (extern "C"), 37 functions: init/shutdown, inputs, LEDs, SC, PD, stats.
src/gpio.*	libgpiod v2: input lines with pull-up or no bias, edge events, outputs. GPIO 7/8 (SPI chip selects) are not touched.
src/adc.*	Two MCP3208 over spidev (/dev/spidev0.0 = CV, 0.1 = knobs), 1 MHz. All 8 channels in one ioctl.
src/inputs.*	1 kHz input loop. Debounce of 8 scans for buttons, no debounce on gates. Encoder decoding identical to encoder.py. Short presses between two snapshots are kept (StretchBit). End-of-loop output (GPIO 16).
src/leds.*	PCA9685 at 0x40 on /dev/i2c-1, 250 Hz. Only changed LEDs are written, in one I2C transaction. Boot/loading animations from bootleds.py run inside the library. Reset LED by software PWM.

File	What it does
src/links.*	ScLink: OSC to sclang with liblo, change-only sending (/neb/hasbeenloaded always sent), listens for /sc/up and /neb/ready. PdLink: TCP text "name value;", connects in the background and retries.
src/nebio.cpp, log.h	C API glue, error messages, logging to stderr as [nebio].
CMakeLists.txt	C++17, pkg-config libgpiod >= 2.0 and liblo, -Wall -Wextra, hidden symbols.

4. Python changes (Code/nebulae/)

File	Change
nebio.py (new)	ctypes wrapper. Loads ../nebio/build/libnebio.so (or env NEBIO_LIB). Same Color/LedDriver interface as leddriver.py. Stops a running bootleds.py before taking over the LEDs.
shiftregister.py, switch.py, MCP3208.py, encoder.py	Rewritten as thin wrappers over nebio. Same classes and methods as before.
scsender.py, pdsender.py	Same ScSend/PdSend API, but OSC and TCP now run in C++. python-osc is no longer used. New: waitSynthIsUp(timeout).
control.py, controlhandler.py	RPi.GPIO and Adafruit SPI removed, nebio used instead. EOL output via nebio. updateAll() logic unchanged.
ui.py	Uses nebio as LED driver; GPIO setup/cleanup removed. LED logic unchanged.
nebulae.py, fileloader.py	launch_bootled/kill_bootled now start and stop the animation inside nebio instead of a separate bootleds.py process.

Unchanged on purpose: bootleds.py, leddriver.py, check_wifi.py, cleanup.py. They run outside nebulae.py (boot, update, wifi, calibration) and still use the old Python drivers.

Scripts

- **Code/scripts/build_nebio.sh** (new): cmake configure + build with -j3 (one core left free for thermal reasons). Incremental, so it is cheap at every boot.
- **Code/scripts/startup.sh**: runs build_nebio.sh (with mountfs rw/ro) before check_wifi, check_calibration and nebulae.
- **.gitignore**: Code/nebio/build/ added.

5. Behaviour changes to watch for

- **Knob smoothing may feel different.** The Python loop is now paced by the snapshot (at most about 500 per second). The filters in AdcData/HybridData smooth per loop, so a different loop speed changes the feel. nebio.stats() shows the real rates.
- **Button debounce is 8 ms** (before: 3 loop iterations).
- **Gates are caught by interrupt**, so a 1 ms trigger is no longer missed.
- **No LED flash at start**: the library switches all LEDs off when it starts.
- **Pure Data** connects in the background and retries; a missing PD only gives one warning.
- If libnebio.so is missing or fails to load, nebulae.py fails with a logged error.

6. How it was compiled and tested on the Windows PC

There is no Nebulae hardware on the PC, so the library was built and tested in **WSL Arch Linux**, the same distribution as on the Nebulae. These builds are x86_64 and only prove that the code compiles and that the logic works; the RPi4 builds its own aarch64 version.

- **Build:** same CMakeLists.txt as on the device, gcc 16. 0 warnings, 37 exported functions.

- **test_logic.cpp:** debounce and encoder decoding compared against transcriptions of the old Python code on long random input (encoder: 305 vs 305 steps), plus the short-press handling. 0 failures.
- **test_links.py:** Python 3.14 + scsender.py/pdsender.py against the built library. OSC types (int, float, string, bool), no resending of unchanged values, resend after setSynthIsDown, /sc/up and /neb/ready, unknown addresses logged, listener restart, PD async connect and exact bytes (b"pitch 0.5;freeze 1;"), and clean failure without /dev/gpiochip0. 22/22 passed.
- **Not testable on the PC:** GPIO, SPI and I2C (real buttons, knobs, LEDs).

Changes to the development PC

- WSL Arch was fully upgraded (229 packages: gcc 16, glibc 2.44, cmake 4.4) and python, libgpiod, liblo and python-reportlab were installed. A harmless getty@.service hook error appeared during the upgrade.
- Three .pacnew files were created, e.g. /etc/pacman.d/mirrorlist.pacnew. The current settings were kept.
- WSL Ubuntu (unused, 1.4 GB) was removed to free disk space.

7. Next: test on the Nebulae

```
sudo pacman -S libgpiod liblo        # libgpiod must be version 2 or newer
sh Code/scripts/build_nebio.sh       # or just reboot: startup.sh builds it
journalctl -u nebulae.service | grep nebio
```

Checklist

- A Csound, a Pure Data and a SuperCollider instrument start and play.
- Buttons, gates (short triggers), knobs, CV inputs and both encoders react correctly.
- LEDs, the loading animations and the reset LED look as before.
- Knob smoothing feels right (otherwise retune the filter coefficients).
- SuperCollider handshake (/sc/up) and PD connection work.
- check_wifi.py and check_calibration.py still work at boot.

8. Step 2 (later)

When step 1 works on the device: the SuperCollider extension, where sclang owns the logic and can drive every LED on the Nebulae, while Python/C++ only reports raw input. Topics: raw input events, an LED claim protocol, heartbeat/handshake, and how instruments declare button combinations. This will be discussed separately.